



Module F - Trusted Computing and Remote Attestation

L2 – Remote Attestation and IMA

Silvia Sisinni

silvia.sisinni@polito.it



**Politecnico
di Torino**

Department of Control and
Computer Engineering



TORSEC

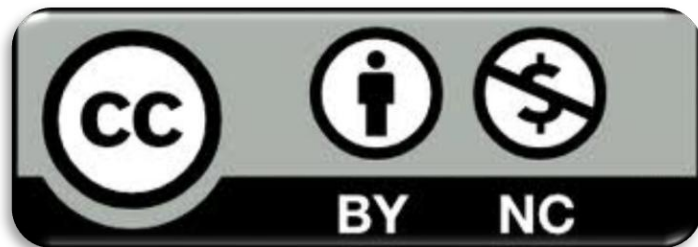
Computer and Network Security Group



License & Disclaimer

License Information

This presentation is licensed under the
Creative Commons BY-NC License



To view a copy of the license, visit:

<http://creativecommons.org/licenses/by-nc/3.0/legalcode>

Disclaimer

- We disclaim any warranties or representations as to the accuracy or completeness of this material.
- Materials are provided “as is” without warranty of any kind, either express or implied, including without limitation, warranties of merchantability, fitness for a particular purpose, and non-infringement.
- Under no circumstances shall we be liable for any loss, damage, liability or expense incurred or suffered which is claimed to have resulted from use of this material.
- This material is used and adapted within the course “Cybersecurity and National Defense”.



From local measurements to remote trust decisions

What we already know

- The platform can measure security-relevant components, starting from a CRTM
- Measurements are extended into TPM PCRs, shielded memory locations updatable only through the extend operation
- PCRs provide a tamper-evident summary of platform state
- Event logs explain what was measured and extended in the PCRs

New question

How can a remote entity use this evidence?

A remote verifier needs to know:

- Is the evidence authentic?
- Is it fresh?
- Does it describe an expected platform state?
- Should this platform be trusted for a specific operation?

What problem does Remote Attestation solve?

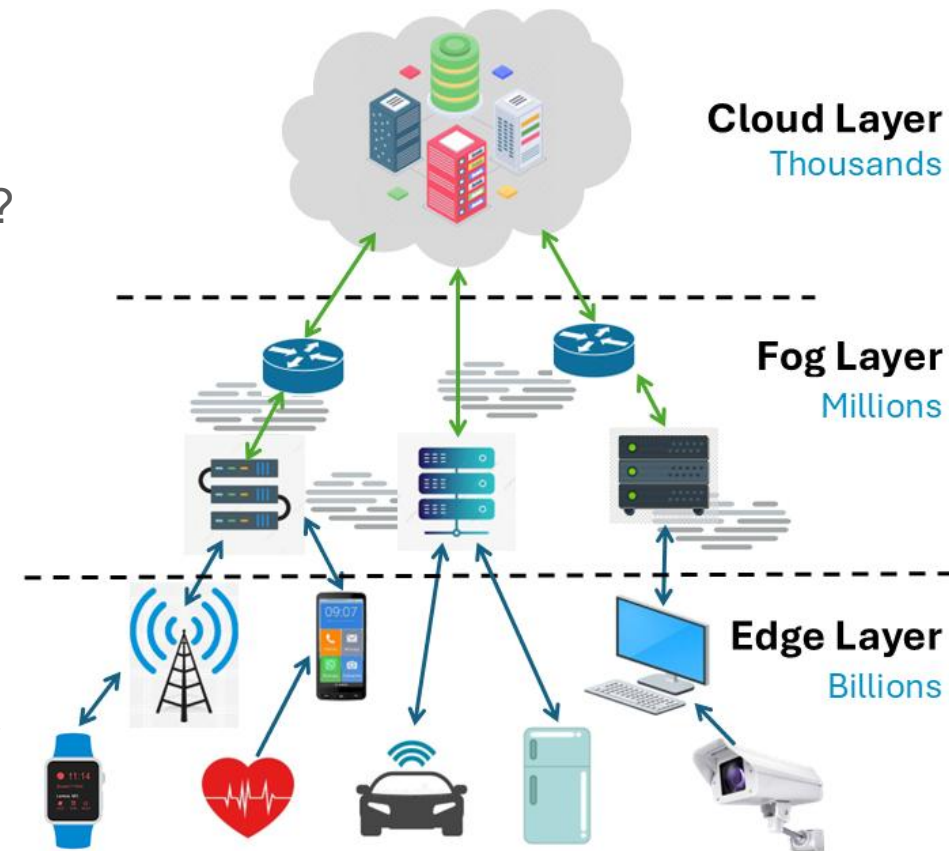
In distributed systems, trust is remote.

A service may need to know:

- Is this node genuine?
- Did it boot with expected firmware and kernel?
- Is Secure Boot enabled and correctly configured?
- Are runtime components still trusted?
- Has the platform been tampered with?

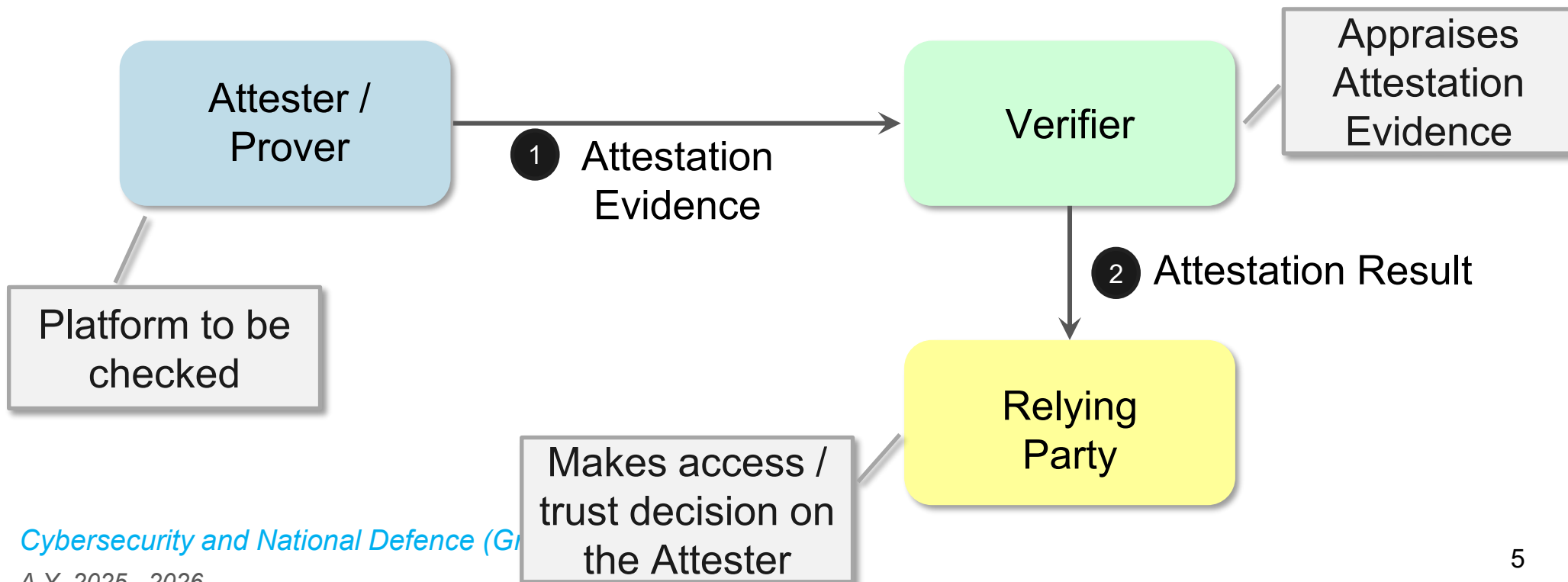
Remote Attestation provides:

- cryptographic evidence of platform state
- freshness against replay attacks
- binding to a hardware or firmware trust anchor
- policy-based trust decisions
- support for Zero-Trust architectures



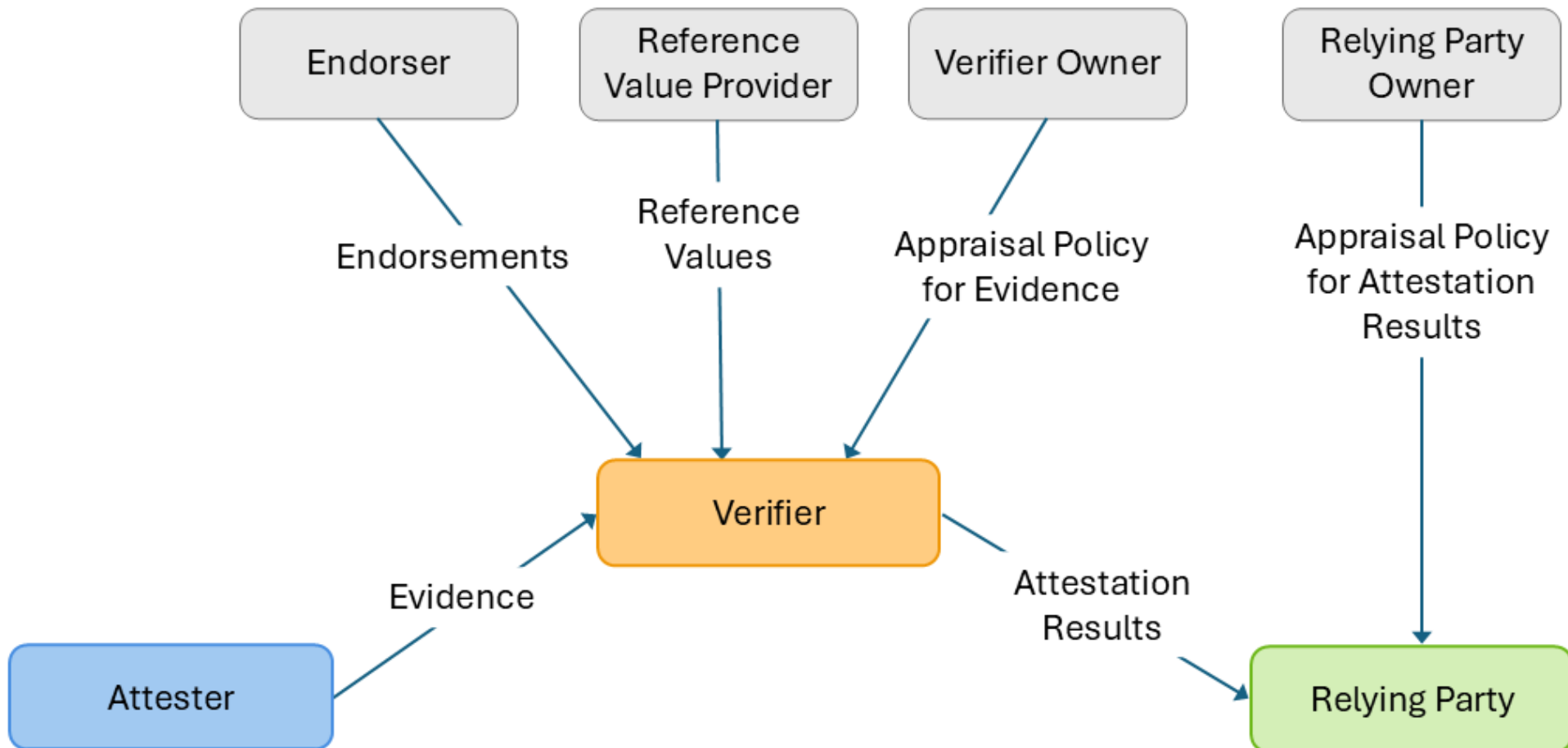
Remote Attestation: basic intuition

Remote Attestation is a process in which a platform provides evidence about its identity and state to another entity, which evaluates whether the platform should be trusted.



Remote Attestation in the RATS architecture

The **IETF Remote ATtestation procedureS (RATS)** architecture generalises Remote Attestation using roles and information flows.





RATS roles: who does what?

Role	Main responsibility
Attester	Creates Evidence about itself and sends it to a Verifier
Verifier	Appraises Evidence, relying on Reference Values, Endorsements and an Appraisal Policies, and produces Attestation Results
Relying Party	Uses Attestation Results to make an access/trust decision
Endorser	Provides Endorsements for trust anchors or attestation keys
Reference Value Provider	Provides known-good or acceptable values (e.g., expected measurements of firmware, boot components, software, or configuration)
Verifier Owner	Configures appraisal policy for Evidence, which determines how Evidence should be evaluated
Relying Party Owner	Configures policy used to interpret Attestation Results



RATS conceptual messages (I)

- **Evidence:** claims about the Attester's identity and integrity state
 - For example, it may include information about which software is running, which firmware was loaded, which configuration is active, or which hardware root of trust is being used.
- **Endorsements:** statements (e.g., certificates) supporting the authenticity of attestation keys or trust anchors
 - They help answer the question: "Is this really a valid attestation-capable platform?"
- **Reference Values:** known-good or acceptable measurements
 - They help answer the question: "Is the state reported by the Attester the state that I expected?"
 - For example, if the Evidence contains a firmware hash, the Reference Value tells the Verifier which firmware hashes are acceptable (e.g., Accepted bootloader hash: A1B2C3, Accepted kernel hash: D4E5F6, ...)



RATS conceptual messages (II)

- **Appraisal Policy:** rules used by the Verifier to interpret the Evidence
 - The policy defines what should be accepted, what should be rejected, and possibly what should be treated as a warning or degraded state
 - For example: Accept the device only if:
 - the bootloader hash matches an accepted Reference Value;
 - the kernel hash matches an accepted Reference Value;
 - Secure Boot is enabled;
 - debug mode is disabled.
 - the Appraisal Policy answers questions like: “Is a mismatch fatal, or only a warning?”, “Can an old firmware version still be accepted?”, “Should the device be denied if debug mode is enabled?”, “Is Secure Boot mandatory for this device?”, ...
- **Attestation Results:** verifier output consumed by a Relying Party, which uses them to make decisions, e.g., granting access, denying access, or applying some restriction

TPM-based Remote Attestation

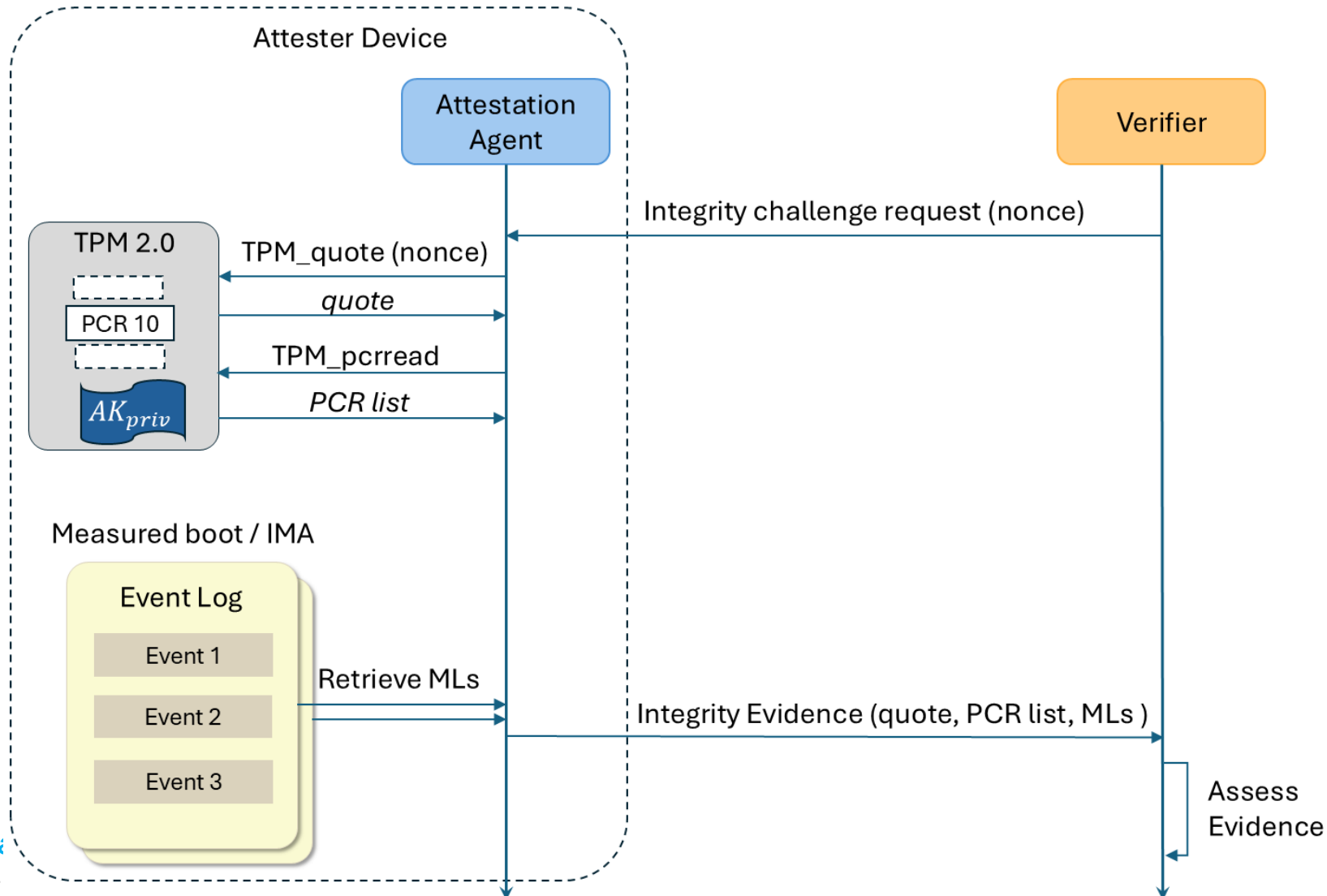
How can a TPM-equipped platform prove its measured state to a remote Verifier?



From RATS to TPM-based attestation

RATS concept	TPM-based attestation example
Attester	Remote platform equipped with a TPM
Evidence	TPM quote + PCR values + measurement logs
Endorsements	EK certificate, AK certificate, TPM manufacturer / Attestation CA certificates
Reference Values	Expected PCR values, golden measurements, approved software digests
Appraisal Policy	Rules defining what is acceptable, e.g., which PCRs must be quoted, what to do when an unknown component is found, ...
Attestation Result	Trusted / untrusted / warning / remediation required

TPM-based integrity challenge protocol





TPM-based integrity challenge protocol

Protocol steps:

1. Verifier sends an integrity challenge with a **nonce** (to ensure freshness of the response)
2. Attestation Agent requests a **TPM quote**, specifying to the TPM the received nonce, the AK to be used and the list of PCRs to be signed
3. TPM signs selected PCRs with the AK, and returns the obtained signature
4. Attestation Agent reads selected PCR values
5. Attestation Agent retrieves measurement logs
6. Attestation Agent sends to the Verifier the collected **Integrity Evidence**, which contains:
 - TPM quote
 - PCR values signed with the quote
 - Measurement logs
 - AK certificate



Why is the nonce needed?

Without freshness, an attacker could replay old “healthy” evidence.

Replay attack idea:

1. Platform is trusted at time t_1
2. Attacker stores a valid quote
3. Platform becomes compromised at time t_2
4. Attacker replays the old quote

Nonce-based protection:

- Verifier sends a fresh unpredictable nonce
- TPM includes the nonce in the signed quote
- Verifier accepts only quotes containing its nonce

✈ Freshness means: this Evidence was produced for this challenge, not copied from the past.

Why do we need an Attestation Key?

Endorsement Key (EK)

- long-term TPM identity key
- certified by TPM manufacturer
- proves that the TPM embedded in the platform is genuine
- privacy-sensitive if used directly
- it is never used as “everyday” signing key for repeated attestation operations, in order not to expose a permanent manufacturer-backed identity

Attestation Key (AK)

- TPM-generated signing key
- Non-migratable key, private part never leaves the TPM
- certified as belonging to a genuine TPM
- used to sign TPM quotes
- operational attestation identity, certified as TPM-resident



The EK anchors trust in the TPM.
The AK is the key used repeatedly for attestation.

How does a Privacy CA certify an AK?

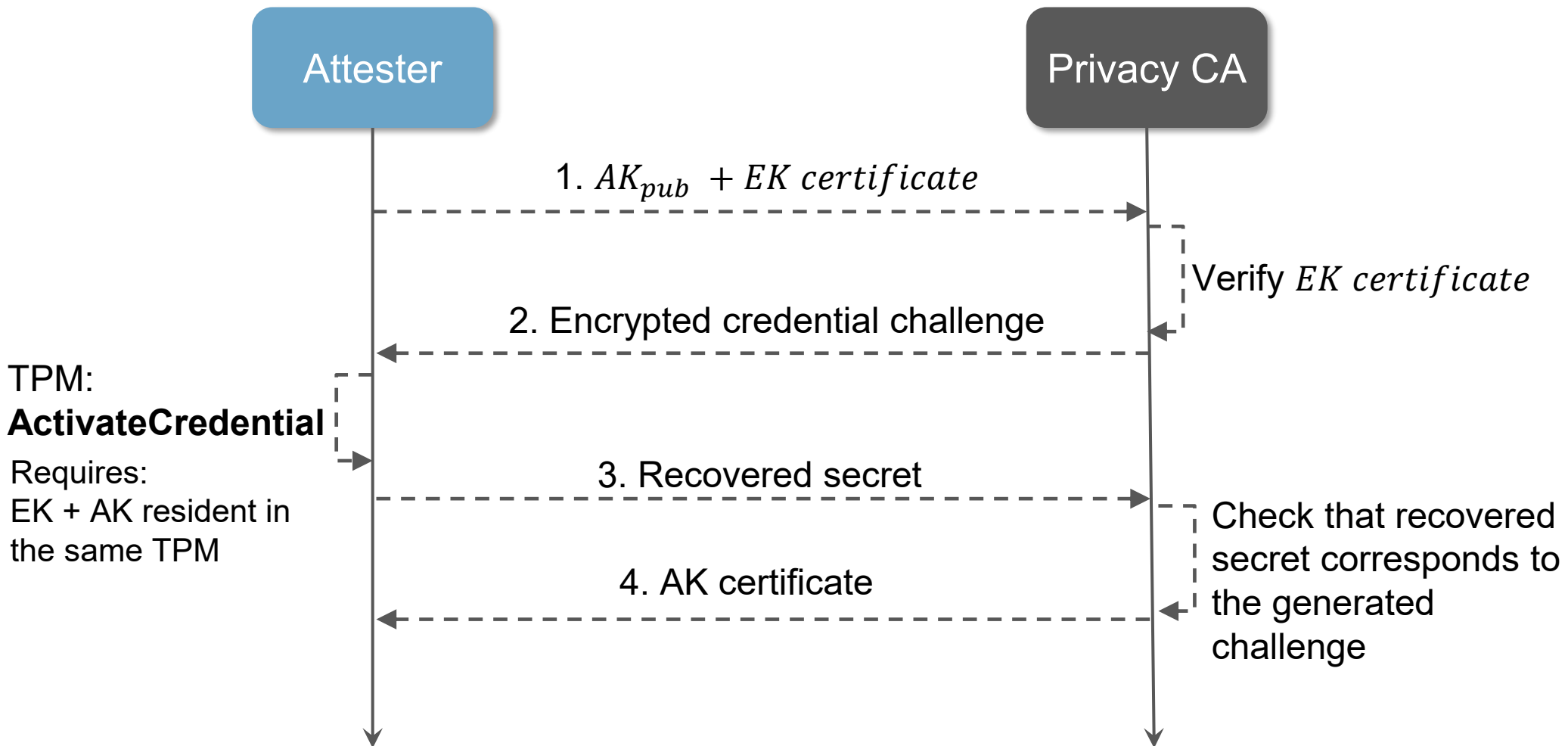
Goal: Certify that an AK is resident in the same TPM as the EK representing a genuine TPM, so that the AK can be used as an identity alias for the TPM.

Simplified protocol:

1. Attester sends to the Privacy CA:
 - AK public key
 - EK certificate
2. Privacy CA verifies the EK certificate, making sure it was issued by trusted TPM manufacturers
3. Privacy CA creates a secret challenge encrypted with the EK_{pub} and bound to the AK
4. TPM runs **ActivateCredential** and recovers the secret only if:
 - it owns the EK private key
 - the AK is loaded/resident in the same TPM
5. Privacy CA receives the secret back and issues an AK certificate

✦ The Privacy CA certifies the AK is resident in a genuine TPM, so future quotes can be verified using the AK certificate instead of exposing the EK.

ActivateCredential protocol





Why “Privacy CA”?

If the EK were used directly

- same long-term TPM identity appears repeatedly
- different services could correlate platform activity
- EK would be exposed to frequent operational use
- poor privacy and poor key-separation practice

With AK + Privacy CA

- EK is used only during AK certification
- AKs become the attestation signing keys
- **different AKs can be used for different contexts**
- verifiers check AK certificates, not the EK directly



The TPM quote

*A **TPM quote** is a TPM-produced signature over selected PCR state and freshness data.*

Inputs:

- selected PCRs
- nonce from the Verifier
- Attestation Key handle/name

Output:

- signed attestation structure:

$$\text{Quote} = \text{Sign}_{AK_{priv}}(\text{selected PCR values} \parallel \text{nonce} \parallel \text{optional metadata})$$

1. proves that selected PCR values is authentic
2. binds the response to the Verifier's challenge



Why the quote alone is not enough

In TPM 2.0, the quote is the **TPM-produced signature over attestation data**.

- It authenticates selected PCR state and binds it to the nonce sent by the Verifier. This gives authenticity and freshness.
- The Verifier also needs the selected **raw PCR values**, retrieved separately, e.g., with: TPM2_PCR_Read
- But PCR values are only aggregates, they could be not enough to understand the platform state:

$$PCR_{final} = Hash(... Hash(Hash(PCR_{initial} \parallel M1) \parallel M2) ... \parallel Mn)$$

- So the Verifier also needs the **measurement log**, containing the detailed history of measurement events, to know:
 - what was measured; in which order measurements were extended;
 - which component generated each measurement; which component caused a mismatch; whether each event is acceptable under policy.

Quote authenticates PCR values. PCRs authenticate the logs. Logs explain history.

Verifier-side appraisal

How does the Verifier transform TPM-based Attestation Evidence into an Attestation Result?



Verifier appraisal pipeline

1. Authenticate AK
↓
2. Verify quote
↓
3. Validate measurement logs
↓
4. Appraise measurements
↓
5. Produce Attestation Result

Goal:

Decide whether the Attester is acceptable for a specific purpose, under a specific policy, at this time.



Step 1: Authenticate the Attestation Key

Goal: The Verifier checks whether the quote-signing key is a legitimate attestation key from the attesting platform, not an arbitrary software-generated key.

Checks may include:

- Does this AK certificate identify the platform I want to attest?
- Was the AK certified as TPM-resident?
- Is the certificate-chain trusted? That is:
 - do I trust the Privacy CA (or Attestation CA) that issued the AK certificate?
 - do I trust the Root CA that certified the Privacy CA, and so on?
 - or (if I'm not using the Privacy CA), do I trust the EK certificate and the TPM manufacturer?
- Is the AK certificate still valid or has it been revoked?

Why this matters:

Without AK authentication, any software key could sign fake evidence, or any platform can impersonate the attesting platform.



Step 2: Verify the quote

The Verifier checks:

1. Signature

- quote signed by the trusted AK (verified at the previous step)

2. Freshness

- nonce in quote matches the nonce sent in the challenge

3. PCR selection

- quote covers the PCRs required by policy (e.g., boot PCRs, runtime PCRs, ...)

4. PCR values

- received PCR values match the quoted PCR state

If any check fails: reject the Evidence.

Step 3: Validate the measurement logs

The **measurement log** is **outside the TPM**, in system memory. Therefore, the **Verifier must check that it has not been tampered with**, so that it **matches the TPM-protected PCR values**.

Verifier replay:

$$PCR_{recomputed} = Extend(Extend(Extend(PCR_{initial} \parallel M1) \parallel M2) \dots \parallel Mn)$$

Then compare:

$$PCR_{recomputed} == PCR_{quoted} \quad ?$$

- **If values match:** log is consistent with the quoted PCR, so it is authentic and can be used for appraisal.
- **If values do not match:** log was modified, truncated, reordered, or does not correspond to the quoted TPM state. $\Rightarrow$ In this case, the Evidence must be rejected.



Step 4: Appraise measurements against policy

After log validation, the Verifier checks individual measurement events.

Reference Values answer:

- Is this firmware measurement known-good?
- Is this bootloader approved?
- Is this kernel version acceptable?
- Is this configuration expected?

Appraisal Policy answers:

- Which PCRs/logs are mandatory?
- Which components are allowed?
- Are unknown components rejected?
- Are debug/test modes acceptable?
- What action follows a failure?



Step 5: Produce the Attestation Result

The Verifier produces an **Attestation Result** for the Relying Party.

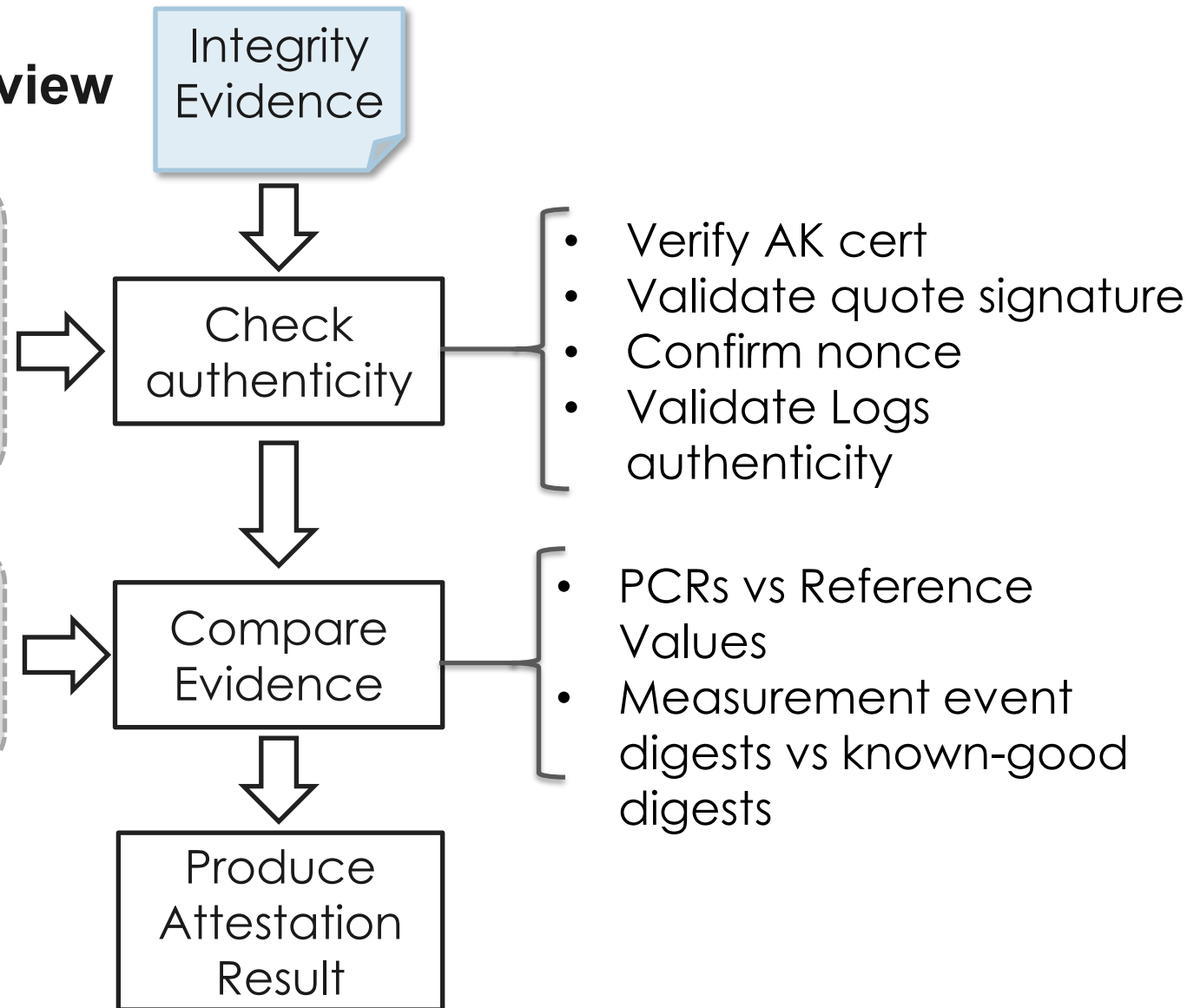
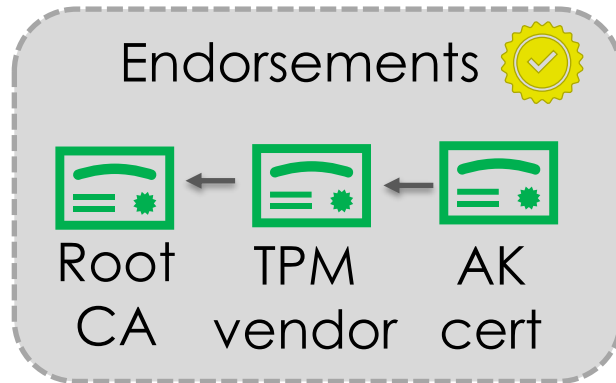
The result may include:

- **trust decision:** trusted / untrusted
- validated claims
- reason for failure (e.g., unknown component detected, or component with known vulnerability detected)
- timestamp / freshness context
- policy used
- recommended action

Example decisions:

- If **trusted**: allow access / deploy workload / release secret
- If **untrusted**: deny connection / quarantine node, trigger remediation (e.g., update vulnerable components, remove / redeploy workload, restart node to restore trusted state, isolate the node and perform in-depth analysis)

Verifier assessment procedure: complete view



Trusted Boot verification

Using Remote Attestation to verify how the platform booted.

Question:

Did the platform start in a trusted environment?

Measured Boot, Secure Boot, Trusted Boot

Concept	Main idea	Decision point
Measured Boot	Records what was loaded	N/A
Secure Boot	Blocks unauthorised boot components. It does not need a TPM.	During boot, on the platform itself
Trusted Boot	Lets a remote Verifier evaluate boot state	After system booted, during remote attestation with a remote entity



Measured Boot records. Secure Boot enforces.
Trusted Boot enables remote verification.



What does the Verifier check during Trusted Boot?

The Verifier may check whether the platform booted with expected:

- firmware / UEFI components
- platform configuration
- boot manager / bootloader
- Secure Boot policy (i.e., was Secure Boot enabled or disabled on the remote platform?)
- kernel and kernel command line
- early drivers / init components

Evidence used:

- boot PCRs (typically PCR[0] – PCR[9])
- measured boot event log
- quote over boot PCRs



Limitation: boot-time trust is not runtime trust

Question:

If a machine booted correctly at 09:00, can we still trust it at 14:00?

A platform may boot in an expected state, but later:

- execute malicious binaries
- load unexpected libraries
- load kernel untrusted modules
- run altered scripts
- use modified configuration files
- start unapproved services



Trusted Boot verifies how the system started.
It does not fully describe what happens after boot.

Runtime integrity with IMA

Question:

If a platform booted in a trusted state, how can we check what happens after boot?

IMA: Integrity Measurement Architecture

What is IMA?

- **IMA** stands for **Integrity Measurement Architecture**
- Linux kernel module, part of the **Linux Integrity Subsystem**
- Introduced in Linux kernel **2.6.30**
- Extends the TCG integrity measurement model into the running Operating System
- Measures selected files before they are used, according to an **IMA policy**

What does IMA add?

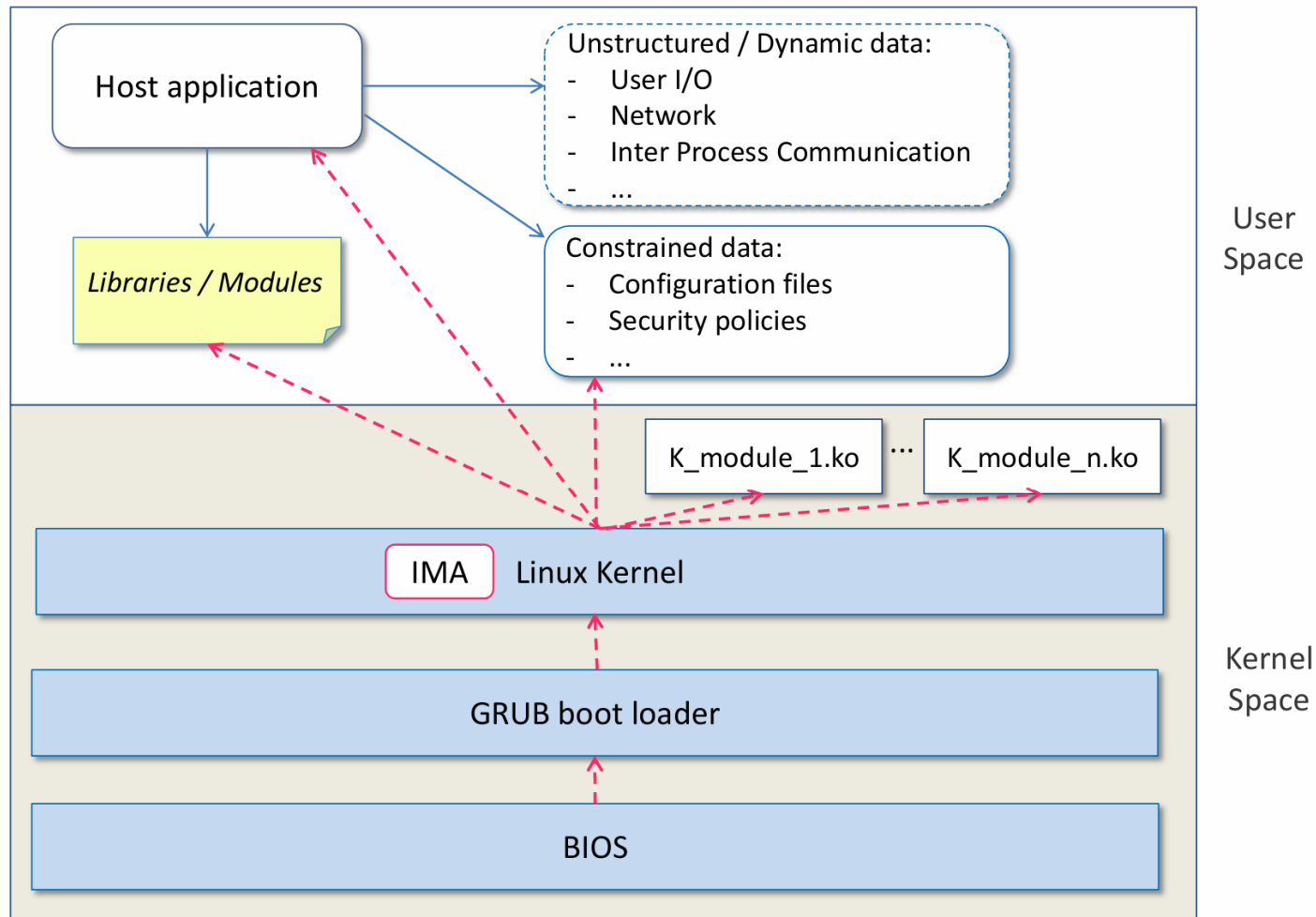
IMA extends the Chain of Trust (CoT) from boot to runtime by measuring:

- kernel modules
- Executables
- shared libraries
- Scripts
- configuration files
- security policies



Measured Boot tells us how the system started.
IMA helps us verify what the Linux system uses after boot.

CoT from the boot up to the application layer



App dependencies



Chain of Trust



What does IMA measure?

High-integrity / structured data

IMA can measure structured security-relevant files:

- Executables / scripts
- shared libraries
- kernel modules
- configuration files
- security policies

The files measured by IMA match a configured **IMA policy**:

- configured through a file in the security file system
- or specified through a kernel boot param

Low-integrity / dynamic data

IMA does not solve all runtime integrity problems:

- user input
- network traffic
- dynamic application data
- runtime application memory after loading
- behaviour not represented by measured files



IMA components

IMA could be conceptually divided into three related components, that can be activated through the IMA policy.

1. IMA Measurement

- measures selected files and stores measurement events in the IMA Measurement Log
- extends event digests into a TPM PCR

2. IMA Appraisal

- checks file integrity locally before allowing access
- compares the current file measurement with trusted metadata stored in security.ima file's extended attributes
- security.ima may contain: a reference hash of the file, or a digital signature over the file hash
- public keys used to verify signatures are stored in trusted kernel keyrings
- deny access if appraisal fails

3. IMA Audit

- records measurement/appraisal events in audit logs

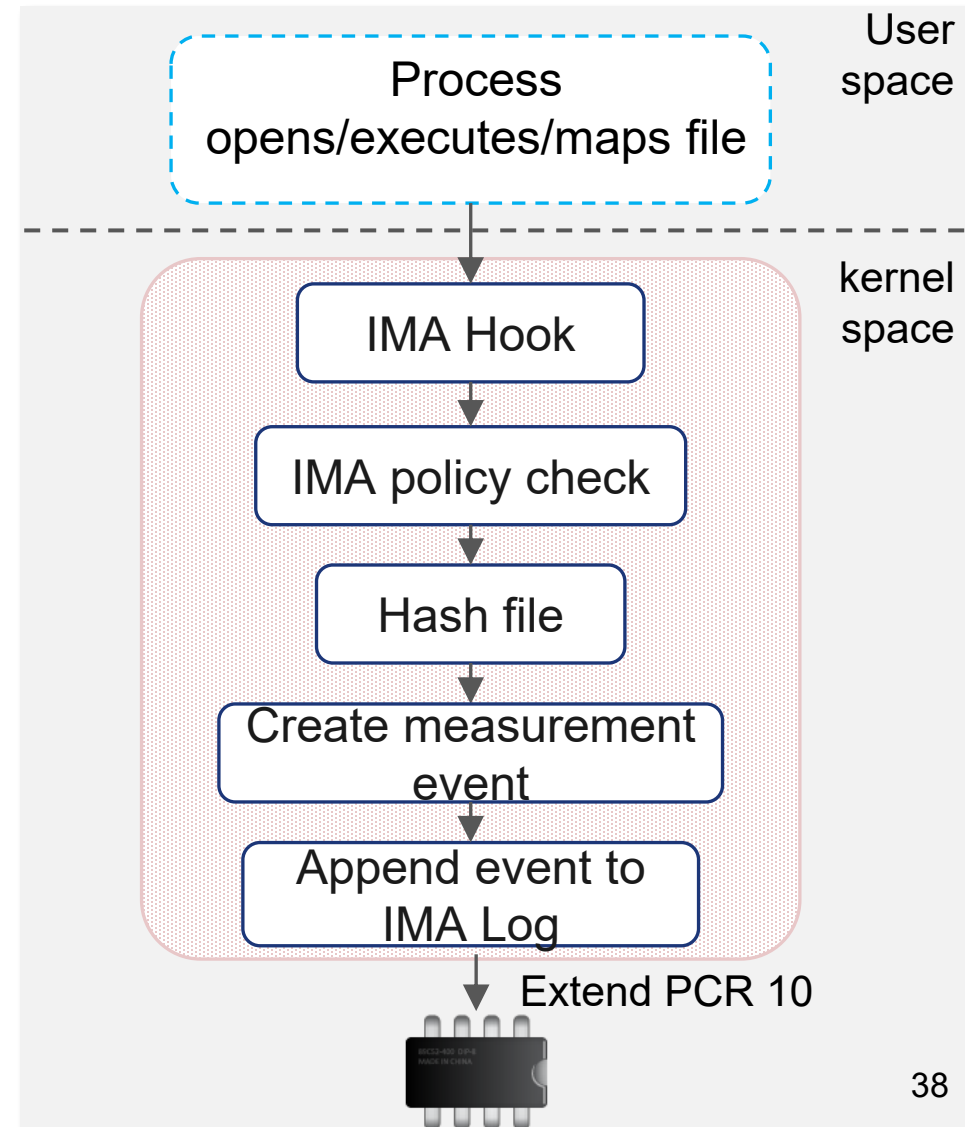
Extended Verification Module (EVM): protects file extended attributes used by IMA

Appraisal

IMA Measurement: from file access to PCR extension

When a file selected by the IMA policy is accessed:

1. Linux Security Module (LSM) / IMA hook is triggered
2. IMA checks the configured policy
3. IMA computes a file digest
4. IMA creates a measurement event (typically, the event contains the file digest and the file path)
5. IMA appends the measurement event to the IMA Measurement Log
6. IMA extends the event digest into a TPM PCR (typically **PCR 10**)



The IMA Measurement Log

- The IMA Measurement Log records runtime measurement events. The metadata recorded for each event depends on the configured IMA template.
- The following is an excerpt from an IMA log created with the “ima-ng” template:

PCR	template-hash	template-name	event data-hash	event name-hint
10	9cab91[...]ab	ima-ng	sha1:c28a[...]8dde	boot_aggregate
10	767f39[...]88	ima-ng	sha1:f29ff4[...]40bf	/usr/bin/kmod
10	4f9fea[...]ec9	ima-ng	sha1:6912d[...]bd8	/usr/lib.../ld-2.31.so
...	...	...	...	...

- **PCR index:** by default PCR 10
- **template hash:** hash computed over the fields after template-name (hash extended in the TPM PCR)
- **template name:** entry format used for the event
- **event data hash:** digest of measured file or event data
- **event name:** file path or event identifier

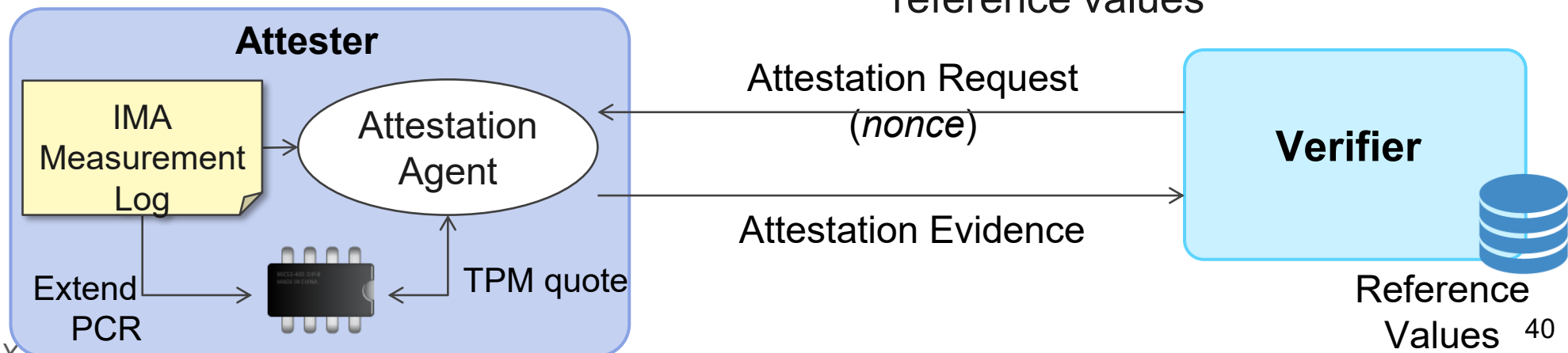
Remote attestation with IMA

Verifier challenges the Attester for obtaining Attestation Evidence, which includes:

- TPM quote over IMA PCR (PCR 10) and boot PCRs (if boot check needed)
- IMA Measurement Log
- boot event log (if boot state is also checked)

Verifier checks:

1. quote authenticity and freshness
2. PCR 10 authenticity
3. resetCount/restartCount in the quote (needed to detect discontinuities between attestations)
4. IMA log consistency with PCR 10
5. individual IMA events against reference values





Verifier-side validation of the IMA log

The IMA log is outside the TPM, so the Verifier must validate it.

Validation procedure:

1. receive quoted PCR 10 value
2. receive IMA Measurement Log
3. start from PCR initial value (typically all zeros)
4. Replay extend operation on all IMA log entries in order
5. recompute PCR 10
6. compare recomputed PCR 10 with quoted PCR 10

$$PCR_{10}recomputed \stackrel{?}{=} PCR_{10}quoted$$

If equal: log is authentic, consistent with TPM state.

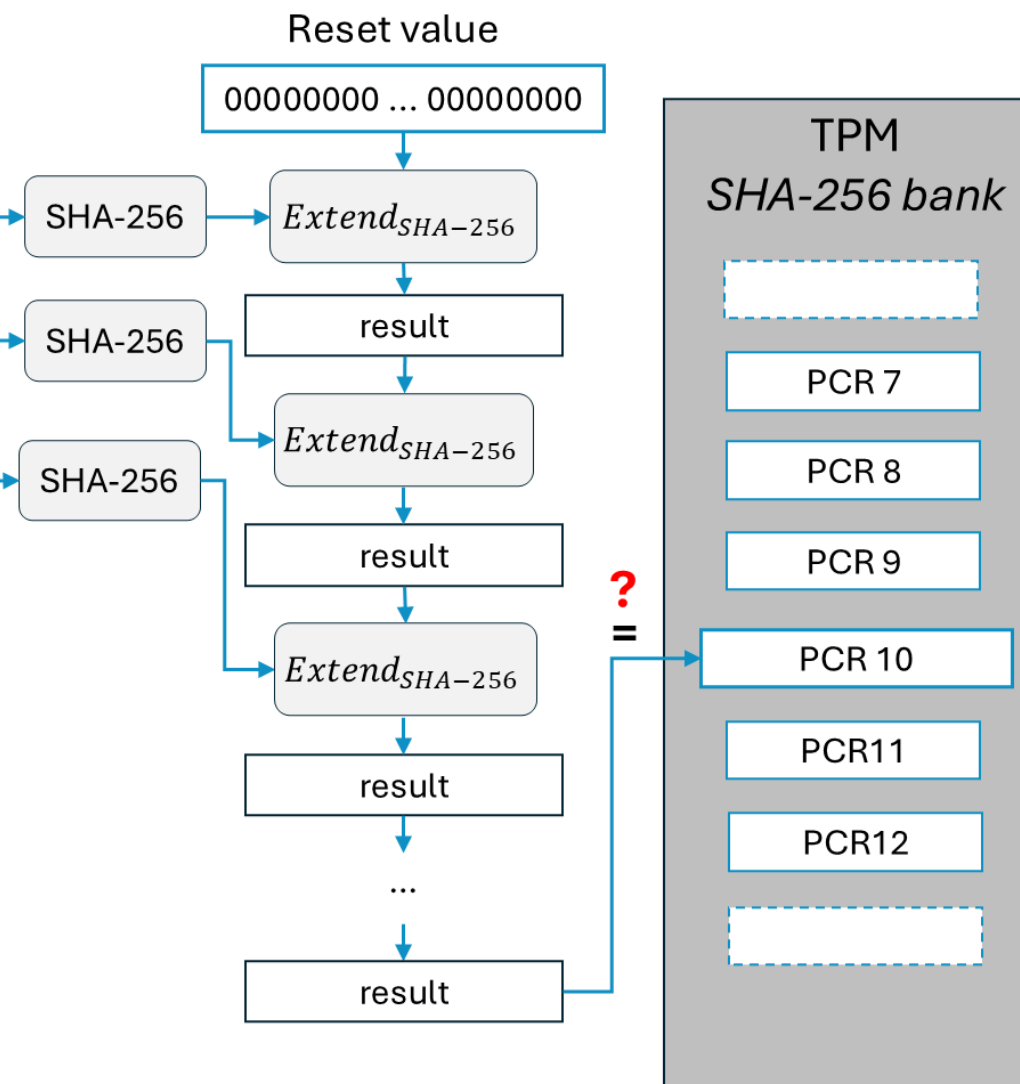
If different: log may be modified, truncated, reordered, or incomplete.



IMA log validation process

IMA Measurement Log

```
10 3ed48c...b71d0 ima-ng sha256:127cb4... boot_aggregate
10 8af71b...aebc ima-ng sha256:3ff76ba2... /init
10 9aa23f0...499dd ima-ng sha256:23e77a1... /bin/sh
10 c710ee...5c490 ima-ng sha256:c164... /etc/ld.so.cache
10 8901dd...47acc ima-ng sha256:1b4aa... /conf/arch.conf
10 76ddb1...dbcc3 ima-ng sha256:3b31c... /usr/bin/rm
10 5acbe...00ac67 ima-ng sha256:3b31c... /usr/bin/cp
...
```

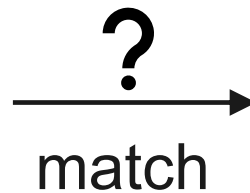


Appraising IMA events against reference values

After validating the authenticity of the IMA log, the Verifier checks each event.

IMA Measurement Event

event name + file digest



Reference values / allowlist

expected digest(s) for
approved files

Possible outcomes:

- file digest matches approved value → trusted
- known file but wrong digest → untrusted
- unexpected file → warning / untrusted



PCR validation proves IMA log integrity; reference-value comparison of each measurement event evaluates platform trustworthiness.

IMA measurement events assessment

IMA Measurement Log

```
10 3ed48c...b71d0 ima-ng sha256:127cb4... boot_aggregate
10 8af71b...aaebc ima-ng sha256:3ff76ba2... /init
10 9aa23f0...499dd ima-ng sha256:23e77a1... /bin/sh
10 c710ee...5c490 ima-ng sha256:c164... /etc/ld.so.cache
10 8901dd...47acc ima-ng sha256:1b4aa... /conf/arch.conf
10 76ddb1...dbcc3 ima-ng sha256:3b31c... /usr/bin/rm
10 5acbe...00ac67 ima-ng sha256:3b31c... /usr/bin/cp
...
```

Reference Values

Event name	Golden values
boot_aggregate	127cb40278dcccab...
/init	3ff76ba2789ccbad... 547aab1b724908ff...
/bin/sh	aab6012ec5398def... 549ddb004ac53319...
...	...



IMA: strengths and limitations

IMA provides

- measurements of files loaded during system runtime
- TPM-anchored integrity evidence of platform after boot
- remote verification of platform integrity state after boot
- detection of unexpected binaries, libraries, scripts, policies, configurations
- integration with TPM-based Remote Attestation challenge-response protocol

IMA does not guarantee

- absence of vulnerabilities
- correctness of measured software
- integrity verification of process memory after loading
- integrity of unstructured dynamic data
- complete coverage of system status unless IMA policy is well configured
- prevention of execution of tampered files unless IMA Appraisal is enforced

Remote Attestation frameworks

From mechanisms to systems

Keylime: TPM-based attestation in practice



What is Keylime?

- open-source Remote Attestation framework
- CNCF-hosted project
- originally developed from research at MIT Lincoln Laboratory
- designed for scalable remote boot attestation and runtime integrity monitoring

What does Keylime use?

- TPM 2.0 as hardware Root of Trust
- TPM quotes over selected PCRs
- measured boot / UEFI event logs for boot-state verification
- Linux IMA Measurement Logs for runtime monitoring
- tenant policies for providing list of PCRs requested in the quote, allowlists/reference values



Keylime turns TPM and IMA evidence building blocks into continuous infrastructure monitoring.



Keylime architecture

Keylime mainly consists of:

1. Agent

- runs on the attested node
- communicates with the TPM
- collects quotes, PCRs, UEFI logs, and IMA logs

2. Registrar

- manages agent enrolment
- stores agent identities and TPM-related public key material
- verifies AK/EK binding during agent registration

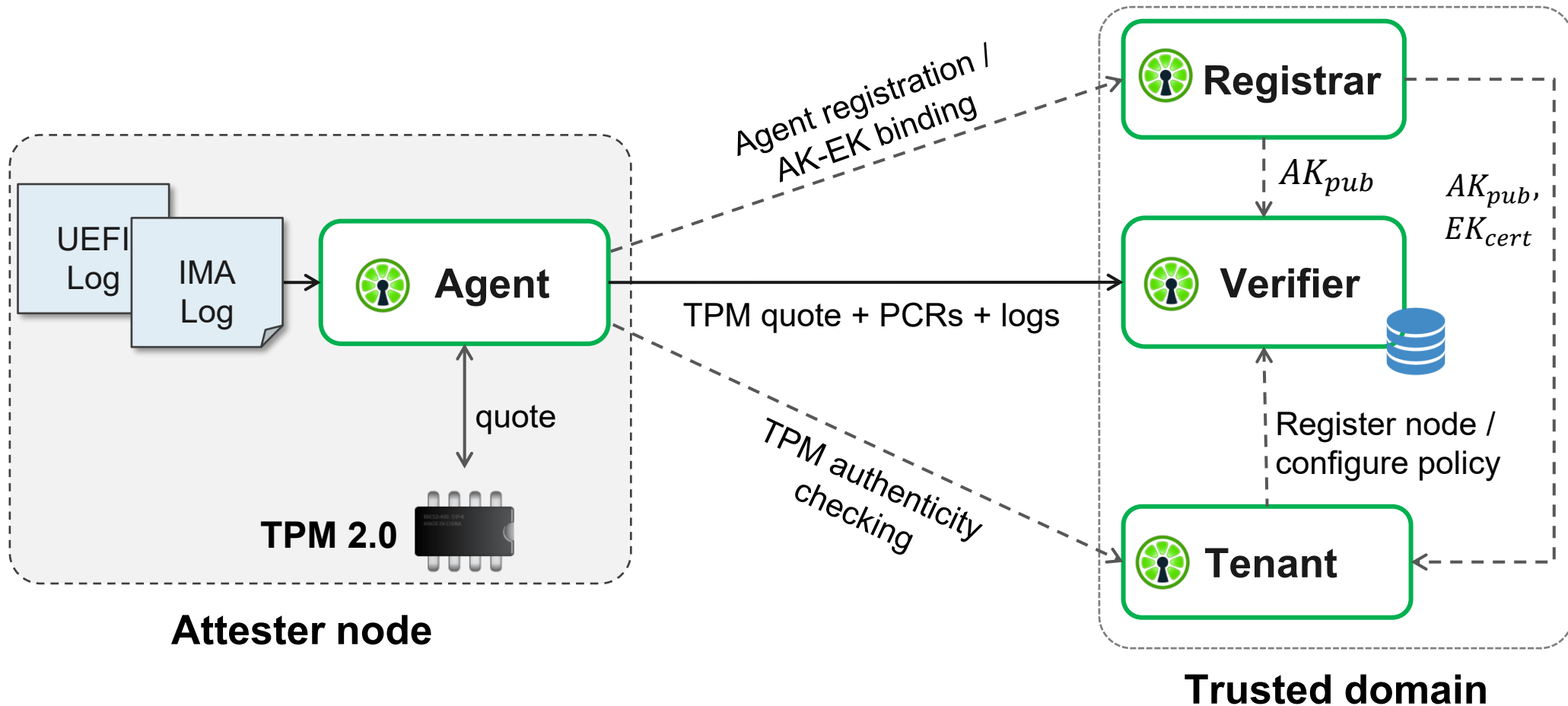
3. Verifier

- continuously attests registered agents
- validates quotes and logs
- sends revocation events if trust fails

4. Tenant

- command-line tool / management interface
- register agents with the verifier, providing the attestation policy
- manages secure payloads provided to the agents only after first successful attestation

Keylime architecture





Secure payload release after first successful attestation

Goal: Release security-sensitive contents only to a node that proves it is in an acceptable state.

Keylime idea: encrypted payload + split decryption key

1. Tenant creates a payload for the remote platform

- e.g., key, certificate, password, configuration file, bootstrap script for deploying workloads

2. Tenant encrypts the payload

- generates a random encryption key and uses it to encrypt the payload
- splits the encryption key into two parts: **U** and **V**

Encryption_Key = random_32_bytes()

V = random_32_bytes()

U = Encryption_Key **XOR** V

Encryption_Key = **U XOR V**



Secure payload release after first successful attestation

3. Before payload provisioning

- Tenant checks the Agent/TPM identity
- e.g., verifies TPM-quote and EK certificate against trusted TPM manufacturers
- this is an identity/trust-anchor check, not yet platform-integrity appraisal

4. Provisioning

- after TPM genuinity verification, Tenant sends to the Agent encrypted payload + **U**
- Tenant sends to the Verifier **V**, together with the attestation policy (e.g., reference values, list of PCRs to be checked)



Secure payload release after first successful attestation

5. After registration of the Agent by the Tenant, the Verifier performs the first real integrity attestation
 - Verifier challenges the Agent
 - checks TPM quote, PCRs, logs, and configured policy
 - **If the node is trusted** (platform state satisfies the configured policy):
 - Verifier sends **V** to the Agent
 - Agent has both key halves and can combine them to recreate the encryption key: $\text{Encryption_Key} = U \text{ XOR } V$
 - Agent decrypts the payload by using the reconstructed encryption key
 - if payload contains a script, Agent executes the script
 - **If the node is untrusted**, V is not released and the payload remains unusable

Runtime monitoring and revocation

After the first successful attestation, the node enters the **continuous monitoring loop**:

1. Verifier periodically challenges the Agent for obtaining a fresh attestation evidence
2. Agent returns fresh evidence: quote, PCRs, boot log, IMA log
3. Verifier validates evidence and policy
4. If evidence remains valid → continue monitoring
5. If evidence fails → trigger revocation framework

Revocation framework

- Notifies other infrastructure nodes that the Agent is untrusted
- allows external systems to react: deny access, isolate node, revoke credentials, remediate

